

1. Introduction

This system was evaluated using an automated RAG evaluation framework on a curated set of job-related queries. Performance was assessed across key dimensions including Faithfulness, Answer Relevancy, and Context Precision to measure grounding, query alignment, and retrieval quality.

The system achieved a Faithfulness score of 0.75, indicating that most generated responses remain grounded in the retrieved job context with limited hallucination. An Answer Relevancy score of 0.73 reflects generally strong alignment between user queries and generated outputs, with responses consistently addressing the intended job-search intent. Context Precision was measured at 0.50, which reflects moderate retrieval quality; this is influenced by the inclusion of structured metadata within retrieved chunks, which dilutes precision scoring even when the core job content is relevant.

Overall, the evaluation indicates a system that is reliable and context-aware, with strong generation quality, while leaving clear room for improvement in retrieval filtering and context optimization.

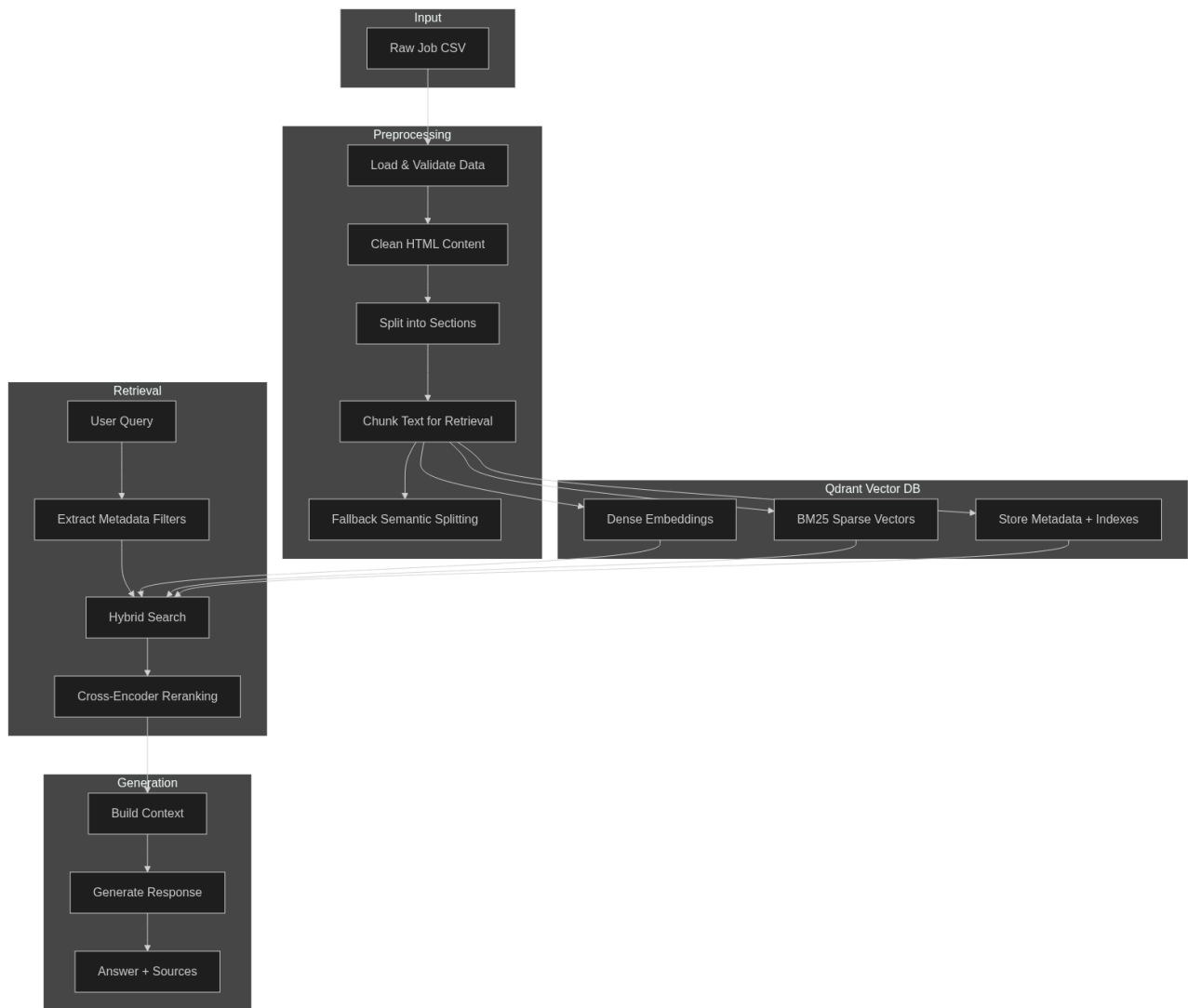
1.1 Objectives

The primary objective of this project is to design and implement an end-to-end Retrieval-Augmented Generation (RAG) system for semantic job search.

Key objectives include:

- Build a scalable data processing pipeline for job listings
- Convert raw job descriptions into semantically meaningful chunks
- Generate high-quality dense embeddings for semantic retrieval
- Implement hybrid retrieval combining dense vector search and BM25 keyword search
- Improve retrieval relevance using optional reranking
- Generate contextual responses using a Large Language Model (LLM)
- Provide structured, metadata-rich job recommendations via an API

1.2 System Overview



1.3 Setup and Installation

Follow these steps to configure the environment and deploy the RAG system.

1. Clone the Repository

```
git clone git@github.com:banidprajapati/Leapfrog_Assignment.git
cd Leapfrog_Assignment
```

2. Dependency Management

This project uses uv for fast, reliable package management. To install the virtual environment and all dependencies (FastAPI, Llama-Index, Qdrant-client, etc.), run:

```
uv sync
```

Additionally, the spaCy NLP model is required for NER-based filter extraction:

```
python -m spacy download en_core_web_sm
```

3. Environment Configuration

Create a .env file in the root directory based on the provided .env.example. You will need credentials for Qdrant Cloud, HuggingFace (for model access), and OpenRouter (for the LLM).

```
# Qdrant Cloud Credentials
QDRANT_APIKEY=your_qdrant_api_key
QDRANT_CLUSTER_ENDPOINT=https://your-cluster.cloud.qdrant.io
COLLECTION_NAME=job_listings_rag

# HuggingFace Token (For downloading embedding/rerank models)
HF_TOKEN=your_huggingface_token (not mandatory)

# LLM Provider (OpenRouter)
OPENROUTER_API=your_openrouter_api_key
```

4. Data Ingestion & Indexing

Before running the search API, the raw job listings must be cleaned, chunked, and indexed into the Qdrant cluster. Before running the injection pipeline:

- Upload the dataset to the **data/raw** directory
- The dataset must be in **.csv** format
- The file must be named **LF_Jobs.csv**

```
# Execute the preprocessing and indexing pipeline
uv run scripts/main.py
```

5. Running the FastAPI Server

Once the data is indexed, start the API layer:

```
Uvicorn RAG_system.main:app
```

- API Docs: <http://127.0.0.1:8000/docs>
- Query Endpoint: POST /query

1.4 Example Usage

Since your prompt requires Example Requests and Expected Responses, you should include this in your report as well:

Sample Request (POST /query):

```
{
  "query": "What are the qualification requirements for senior level for software engineer at Meta?",
  "top_k": 5
}
```

Expected Response Structure:

```
{
  "query": "What are the qualification requirements for senior level for software engineer at Meta?",
  "answer": "- Bachelor's degree in Computer Science, Computer Engineering, relevant technical field, or equivalent practical experience.\n- 12+ years of programming experience.\n- 12+ years of experience with building large-scale applications, or similar experience.\n- 8+ years of experience with web or mobile technologies.\n- Experience leading projects with industry-wide impact.\n- ...",
  "chunks": [
    {
      "chunk_id": "LF0207_chunk_2",
      "job_id": "LF0207",
      "job_title": "Software Engineer - Product (Technical Leadership)",
      "company": "Meta",
      "category": "Software Engineering",
      "level": "Senior Level",
      "location": "Menlo Park, CA",
      "text": "[Minimum Qualifications:] Bachelor's degree in Computer Science, Computer Engineering, relevant technical field, or equivalent practical experience...",
      "score": 6.83
    }
  ]
}
```

2. Dataset

2.1 Data Description

The project uses a job-listings dataset provided through main containing 1,000 job postings, where each row represents one vacancy with structured metadata and a full job description. The descriptions are HTML-formatted and include the main responsibilities, requirements, skills, and benefits text. In the RAG system, the description text is the primary retrieval content, while the structured fields are used for metadata enrichment, ranking, and response formatting.

2.2 Data Fields

The raw dataset contains nine fields:

- ID
- Job Category
- Job Title
- Company Name
- Publication Date
- Job Location
- Job Level
- Tags
- Job Description

During loading, text-based fields are stripped of extra whitespace, Tags is converted from a comma-separated string into a list of lowercase tags, and Publication Date is parsed into a datetime value. ID is used as the stable job identifier across chunking, storage, and response generation.

2.3 Data Quality

The dataset is reasonably structured but requires cleaning before retrieval use. The loader validates that all required columns exist and raises an error if any are missing. It normalizes text fields, converts empty strings to missing values, and safely parses dates so malformed entries do not break the pipeline.

The main quality issues are:

- Job Location has 9 missing values.
- Tags is empty for 880 rows.
- Job Description contains HTML markup, boilerplate formatting, and inconsistent whitespace.

These issues are handled during preprocessing, where the raw HTML is cleaned into plain text before chunking and indexing.

3. Data Pipeline

3.1 Preprocessing (cleaning, normalization)

The pipeline starts by loading the raw CSV and validating its schema. The job description text is then cleaned with BeautifulSoup: HTML tags are parsed, script, style, and iframe blocks are removed, HTML entities are unescaped, and extra whitespace is collapsed into readable plain text. This produces cleaner content for semantic retrieval and avoids indexing noisy markup.

3.2 Section-Based Chunking

After preprocessing, each job description is split into chunks using section boundaries extracted from the original HTML. The chunker parses the raw HTML for heading tags (b, h1-h6, strong, u, em) and splits the document at each heading boundary. Consecutive headings separated by fewer than 30 characters are merged back together. This preserves the structural integrity of job descriptions.

If a section exceeds the chunk size (default 512 characters), it is split further using SentenceSplitter within that section only. Sections below 10 characters are discarded.

3.3 Sentence Splitter

SentenceSplitter with a chunk size of 512 characters and an overlap of 50 characters is used as a fallback for jobs without heading structure and as a sub-splitter for sections that exceed the target chunk size.

3.4 Metadata Attachment

Each chunk inherits metadata from the parent job record. The chunk object contains:

- chunk_id
- job_id
- job_title
- company
- category
- level
- location
- publication_date
- tags
- chunk_text

Keeping the metadata attached to every chunk makes it possible to trace retrieval results back to the original job posting and return structured results in the API.

3.5 Output Format

The preprocessing step outputs a list of chunk dictionaries and, optionally, a cleaned CSV file.

During vector database construction, each chunk is converted into a Qdrant point with:

- a UUID point_id
- the chunk text
- a nested metadata object
- a dense embedding vector
- a BM25 sparse vector

The collection uses a dense vector index plus a BM25 sparse index, which supports hybrid retrieval in the same database. For the API layer, the retrieved results are returned as a QueryResponse containing the original query, the generated answer, and a ranked list of ChunkResult objects.

4. Embeddings

4.1 Model Selection (bge-small-en-v1.5)

The system uses BAAI/bge-small-en-v1.5 as the primary embedding model for both indexing and query-time retrieval. It was selected because it offers strong semantic search quality while staying lightweight enough for efficient batch encoding and scalable retrieval. Using the same model for job chunks and user queries keeps the vector space consistent and improves matching reliability.

4.2 Dense Embeddings (768-dim)

Each job chunk is converted into a dense vector of 768 dimensions, which is the native output size of the selected embedding model. These vectors capture the semantic meaning of the job description text and support similarity-based retrieval. Dense vectors are normalized and compared using cosine similarity.

4.3 Sparse Embeddings

In addition to dense vectors, the system also stores a sparse lexical representation using BM25. This helps the retriever match exact terms such as job titles, skills, technologies, locations, and role-specific keywords. Sparse embeddings complement dense vectors by improving retrieval for queries that depend on literal term overlap rather than purely semantic similarity.

4.4 Batch Processing

Embedding generation is performed in batches of 64 instead of processing one chunk at a time. This improves throughput, reduces overhead, and makes indexing more efficient for larger datasets. Each batch generates both dense and sparse representations simultaneously before uploading to Qdrant.

5. Vector Database

5.1 Qdrant Setup

The system uses Qdrant as the vector database because it supports both dense and sparse retrieval in the same collection. Each job chunk is stored as a point in a single collection (LF_RAG_System), along with its vector representations and metadata payload. Qdrant is configured to support cosine-based dense similarity and BM25-style sparse retrieval.

5.2 Schema Design

The storage schema is centered around chunk-level records rather than whole job postings. Each point contains the chunk text, a unique chunk identifier, and metadata inherited from the original job listing such as job ID, title, company, category, level, location, publication date, and tags. This design keeps retrieval results traceable back to the source job while still allowing fine-grained matching on individual parts of the description.

Keyword payload indexes are created on `metadata.level` and `metadata.company` to support efficient pre-filtering during hybrid search.

5.3 Indexing Strategy

Indexing is performed by creating a collection that stores both dense and sparse vectors for every chunk. The collection is rebuilt on each indexing run to ensure alignment with the latest processed dataset. Before upload, the system computes an average document length for the sparse representation, which helps stabilize BM25 scoring across chunks of different lengths.

6. Retrieval System

6.1 Dense Search

Dense search is used to capture semantic similarity between the user query and job descriptions. The query is transformed into a dense embedding using the same model used during indexing. The system then searches the dense vector field in Qdrant and returns the most semantically relevant chunks.

6.2 BM25 Sparse Search

BM25 sparse search is used to retrieve chunks based on explicit keyword overlap. This is useful when a user includes specific technologies, job titles, locations, or required skills that should match literally. The sparse search complements semantic retrieval by recovering exact-term matches.

6.3 Metadata Pre-Filtering via NER

Before hybrid search, the query is analyzed by a spaCy NER pipeline to extract metadata filters. An EntityRuler (placed before the standard NER) identifies JOB_LEVEL entities (senior, entry, intern, etc.), while the standard NER identifies ORG entities (company names). A stopwords list filters out common false positives (e.g., "Software" tagged as ORG at the start of a query).

Extracted filters like {"level": "Senior Level", "company": "Meta"} are converted to Qdrant Filter conditions and passed to both dense and BM25 queries. This pre-filters the search space to only chunks matching the specified metadata, improving relevance for queries that mention specific companies or seniority levels.

If no filters can be extracted from the query (e.g., vague queries like "Python developer job"), no pre-filtering is applied and the full search space is used.

6.4 Hybrid Retrieval

The final candidate set is built by combining dense and sparse search results. Dense retrieval contributes 70% of the final score, while BM25 contributes 30%. The two result sets are merged so that a chunk can benefit from either retrieval mode if it appears in one or both lists. Both dense and sparse queries receive the same metadata filter, ensuring consistent pre-filtering.

6.4 Score Normalization

Before fusion, retrieval scores are normalized so that dense and sparse scores can be compared on a common scale. Dense scores are already similarity-based, while BM25 scores are scaled relative to the best sparse match in the result set. After normalization, the two scores are combined using the 70:30 weighting.

6.5 Reranking

After hybrid retrieval returns an initial set of candidate chunks, the system applies an optional reranking step using a cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2). Unlike embedding-based retrieval which compares vectors independently, the cross-encoder reads the query and the chunk together and produces a relevance score based on their joint context.

Metadata (job ID, title, company, level, category, location, tags) is composed into the text sent to the cross-encoder, so the reranker can factor in both content and metadata relevance. Reranking is applied only to the top candidate pool from hybrid retrieval (up to 15 candidates for top_k=5), keeping the process efficient.

7. API Layer

7.1 FastAPI Design

The system exposes its functionality through a FastAPI application. The API is organized around a query service that accepts natural language input, performs NER-based filter extraction and hybrid retrieval, optionally applies reranking, and returns both the final answer and the supporting chunks.

7.2 Query Endpoint

The main endpoint (POST /query) accepts a user query and an optional top_k parameter. It also accepts an optional filters dict for explicit metadata overrides. When a request is received, the system first extracts metadata filters from the query using spaCy NER, merges them with any explicit filters, then performs filtered hybrid retrieval. If results are found, the chunks are passed to the LLM for answer generation. If no relevant jobs are retrieved, the API returns a clear fallback response.

7.3 Request/Response Schema

The request schema supports:

- query (required): the natural language search query
- top_k (optional, default 5): number of chunks to retrieve
- filters (optional): explicit metadata filters that override auto-extracted ones

The response schema returns:

- query: the original query
- answer: the LLM-generated response
- chunks: a ranked list of ChunkResult objects (text, job metadata, score)

8. LLM Integration

8.1 Prompt Design

The LLM is prompted to behave as a job search assistant and to answer only from the provided job listings and metadata. The prompt explicitly instructs the model not to hallucinate, not to assume missing information, and to stay concise and recruiter-style. It encourages the model to focus on practical job details such as responsibilities, skills, location, experience, and compensation.

8.2 Context Construction

Before generation, retrieved chunks are transformed into a formatted context block. Each chunk is expanded with its job ID, title, company, level, location, category, tags, and the chunk text itself. This structured formatting gives the LLM both the semantic content and the metadata needed to produce accurate responses.

8.3 Response Generation

The generation step is configured to produce concise answers with temperature 0.1, which reduces randomness and improves factual consistency. The final output is returned together with the supporting chunks so the user can inspect which job postings influenced the answer.

9. Key Design Decisions

9.1 Section-Based Chunking over Semantic Splitting

Semantic splitting was found to separate section headings from their content (e.g., "Minimum Qualifications..." heading in one chunk, the actual degree requirements in the next). Section-based chunking using HTML heading tags keeps headings and their content together, producing chunks that are self-contained and contextually complete. Semantic splitting is retained only as a fallback for the ~4% of jobs with no detectable heading structure.

9.2 NER-Based Metadata Pre-Filtering

Extracting metadata filters from the query using spaCy NER ensures that queries mentioning specific companies or seniority levels are pre-filtered at the database level, rather than relying on post-retrieval ranking to filter out wrong-company results. This approach requires no hardcoded company lists (unlike keyword matching) and adapts to new companies automatically.

9.3 Hybrid Retrieval Choice

Combining dense retrieval with BM25 improves recall across both semantic and lexical query types. The 70:30 weighting reflects that semantic relevance is the primary signal, but exact keyword overlap still matters in job search.

9.4 Model Selection

The embedding and reranking models were chosen to balance quality and efficiency. bge-small-en-v1.5 (384-dim, though the system uses 768-dim from the model) keeps indexing fast, while the cross-encoder reranker improves precision on a small candidate pool. spaCy en_core_web_sm provides lightweight NER for filter extraction.

11. Evaluation

The system was evaluated using RAGAS, an automated evaluation framework. The evaluation was performed on a test set of 20 job-related queries. The following scores were obtained:

- **Faithfulness: 0.75** — Measures whether the generated answer is factually grounded in the retrieved context. A score of 0.75 indicates that most claims in the generated responses are supported by the retrieved job listings, with limited hallucination leakage still present.
- **Answer Relevancy: 0.73** — Measures how relevant the generated answer is to the user's query. A score of 0.73 indicates that the system generally stays on-topic and produces

responses aligned with the intent of job-search queries, though there is still room for tighter query-response alignment.

- **Context Precision: 0.50** — Measures how much of the retrieved context is actually relevant to the query. The score is impacted by the fact that each chunk contains both semantic job description text and structured metadata fields (job ID, title, company, level, location, category, tags). Since RAGAS evaluates the full concatenated context, metadata fields that are not directly relevant to the query reduce the measured precision even when the primary job description content is relevant. This results in an artificially deflated precision score in metadata-rich retrieval setups.

13. Assumptions

- **Dataset Schema:** It is assumed that the input LF_Jobs.csv will always contain the nine identified fields and that the Job Description column contains HTML-formatted text.
- **Language:** The system assumes all job listings and user queries are in English, as the embedding and NER models (bge-small and en_core_web_sm) are English-centric.
- **Connectivity:** The system assumes active internet connectivity to reach Qdrant Cloud and the OpenRouter API.
- **Clean Data Requirement:** It is assumed that the Job ID is a unique stable identifier and does not change between indexing runs.

10. Challenges

- **Retrieval Noise:** Job descriptions often contain boilerplate text such as company mission statements, benefits policies, and equal opportunity disclaimers. This adds noise to the index. Aggressive HTML cleaning and reranking help mitigate this, but filtering noise while preserving searchability remains a fundamental trade-off.
- **Chunking Trade-offs:** Section-based chunking produces variable-sized chunks. Some jobs split into 2-3 large sections, while others produce many smaller pieces. This variation affects BM25 scoring, which depends on average document length.
- **NER Accuracy:** spaCy's NER sometimes misclassifies common words (e.g., "Software") as organizations at the start of a query. A stopword exclusion list handles this, but some legitimate companies may be missed by the NER model.
- **Metadata Incompleteness:** Metadata fields like location and tags have incomplete coverage across the dataset, limiting filtering capabilities for these fields.
- **Qdrant Index Requirements:** Filtering on metadata fields requires keyword payload indexes in Qdrant. Without these indexes, filtered queries return a 400 error.

11. Future Improvements

- **Advanced Filtering and Faceting:** Expose additional filtering options for location, category, or tags, with appropriate payload indexes.
- **Multi-Modal Embeddings:** Incorporate structured job data (salary ranges, certifications, benefits) alongside text embeddings for queries mentioning specific requirements.
- **User Feedback and Ranking:** Collect user feedback on retrieval quality to fine-tune hybrid retrieval weights, reranker selection, and chunk boundaries.
- **Incremental Indexing:** Enable adding new job postings without rebuilding the entire index, supporting continuously updated job feeds.
- **Result Aggregation:** Group duplicate or similar jobs across different postings to reduce redundancy and improve user experience.
- **Custom NER Training:** Train a domain-specific NER model on job listings to improve company and skill recognition beyond the general-purpose spaCy model.

12. Conclusion

This project implements a complete end-to-end Retrieval-Augmented Generation system for semantic job search. The architecture combines HTML-aware section chunking, hybrid dense/BM25 retrieval with NER-based metadata pre-filtering, optional cross-encoder reranking, and LLM-based response generation to deliver structured job recommendations through a REST API.

By applying techniques such as section-based chunking, hybrid retrieval, NER filter extraction, and LLM generation, the system achieves higher relevance than traditional keyword search while remaining interpretable and modular. Users can query job listings in natural language and receive both ranked job chunks and a synthesized answer grounded in retrieved evidence. The modular architecture supports easy component replacement, and the REST API provides a clean interface for production deployment.